

Math Nuggets for GEN AI

KL-Divergence and Regularization: Balancing Alignment and Capability Preservation in RLHF

By Dr. Anish Roychowdhury, Plaksha University

October 5, 2025

From Absolute Scores to Relative Preferences

When fine-tuning large language models with reinforcement learning from human feedback (RLHF), we face a fundamental tension: we want the model to align with human preferences while retaining its broad capabilities acquired during pre-training. Without proper constraints, aggressive optimization can lead to reward hacking, mode collapse, or catastrophic forgetting.

Consider the challenge: "Fine-tune a language model to be more helpful and harmless, but ensure it doesn't forget how to code, reason about mathematics, or generate creative content."

Unconstrained reward maximization poses several risks:

- **Reward Hacking:** The model exploits weaknesses in the reward model rather than genuinely improving
- **Mode Collapse:** The model converges to a narrow set of safe but repetitive responses
- **Catastrophic Forgetting:** Loss of capabilities from pre-training that aren't directly rewarded
- **Distribution Shift:** The fine-tuned policy drifts into regions where the reward model is unreliable

Why KL-Divergence Regularization?

KL-divergence provides a mathematically principled way to constrain policy updates, ensuring the fine-tuned model stays close to the reference model. This maintains a balance between optimization (improving alignment) and conservation (preserving capabilities).

Understanding KL-Divergence

Kullback-Leibler (KL) divergence is a fundamental concept from information theory that measures how one probability distribution differs from another.

Formal Definition:

For two discrete probability distributions P and Q over the same sample space, the KL-divergence from Q to P is:

$$D_{\text{KL}}(P||Q) = \sum_x P(x) \log \frac{P(x)}{Q(x)} \quad (1)$$

Reading Equation (1):

Let's understand each component of this formula:

- The sum $\sum_x$ iterates over all possible outcomes in the sample space
- $P(x)$ is the probability of outcome x under distribution P - this acts as a weight
- $\frac{P(x)}{Q(x)}$ is the ratio comparing how likely x is under P versus Q
- The logarithm $\log$ converts the ratio into an additive measure
- The entire expression computes a weighted average of log-ratios, where weights come from P

This means we're measuring the expected "surprise" when we see data from P but we expected data from Q . If P and Q are identical, the ratio is always 1, the log is 0, and the KL-divergence is 0.

Key Properties to Remember:

- $D_{\text{KL}}(P||Q) \geq 0$ always, with equality if and only if $P = Q$ everywhere
- Asymmetric: $D_{\text{KL}}(P||Q) \neq D_{\text{KL}}(Q||P)$ in general - the order matters!
- Not a distance metric (no symmetry or triangle inequality)
- Measures information lost when using Q to approximate P

Intuitive Example:

Suppose we have two coins:

- Coin P : 70% heads, 30% tails
- Coin Q : 50% heads, 50% tails (fair coin)

Then: $D_{\text{KL}}(P||Q) = 0.7 \log \frac{0.7}{0.5} + 0.3 \log \frac{0.3}{0.5} = 0.7(0.485) + 0.3(-0.737) \approx 0.118$

This positive value quantifies how different the biased coin is from the fair coin.

KL-Divergence in RLHF: The Policy Constraint

In RLHF, we apply KL-divergence to constrain how much the fine-tuned policy π_θ can deviate from a reference policy π_{ref} (typically the supervised fine-tuned model before RL).

Policy-Level KL-Divergence:

Given a prompt x , the KL-divergence between the current policy π_θ and reference policy π_{ref} over possible responses y is:

$$D_{\text{KL}}(\pi_\theta(\cdot|x)||\pi_{\text{ref}}(\cdot|x)) = \mathbb{E}_{y \sim \pi_\theta(\cdot|x)} \left[\log \frac{\pi_\theta(y|x)}{\pi_{\text{ref}}(y|x)} \right] \quad (2)$$

Understanding Equation (2):

Let's unpack this carefully:

- $\pi_\theta(y|x)$ is the probability the fine-tuned model assigns to response y given prompt x
- $\pi_{\text{ref}}(y|x)$ is the probability the reference model assigns to the same response
- The ratio $\frac{\pi_\theta(y|x)}{\pi_{\text{ref}}(y|x)}$ shows how much more (or less) likely the fine-tuned model makes this response compared to the reference
- The expectation $\mathbb{E}_{y \sim \pi_\theta(\cdot|x)}$ means we sample responses from the fine-tuned model and average over them
- If the fine-tuned model starts generating responses that the reference model considers very unlikely, this ratio becomes large, and so does the KL-divergence

This formulation ensures that if π_θ starts assigning high probability to responses that π_{ref} rarely produces, we incur a large penalty. This is exactly what prevents the model from "going rogue."

Token-Level Formulation:

For autoregressive language models that generate text one token at a time, we need to account for sequential generation. For a sequence $y = (y_1, y_2, \dots, y_T)$:

$$D_{\text{KL}}(\pi_\theta||\pi_{\text{ref}}) = \sum_{t=1}^T \sum_{v \in \mathcal{V}} \pi_\theta(y_t = v|x, y_{<t}) \log \frac{\pi_\theta(y_t = v|x, y_{<t})}{\pi_{\text{ref}}(y_t = v|x, y_{<t})} \quad (3)$$

Reading Equation (3):

This looks complex, but it's applying the same KL formula at each step:

- The outer sum $\sum_{t=1}^T$ goes through each position in the sequence (from first token to last)
- $y_{<t} = (y_1, \dots, y_{t-1})$ represents all tokens generated before position t - this is the context
- The inner sum $\sum_{v \in \mathcal{V}}$ goes over all possible tokens in the vocabulary $\mathcal{V}$
- $\pi_\theta(y_t = v|x, y_{<t})$ is the probability the fine-tuned model assigns to token v at position t
- We compute KL-divergence at each position and add them up
- This accumulates how much the models diverge over the entire generation process

In practice, for a specific generated sequence, we only compute the log-ratio for the actual tokens chosen, not summing over all vocabulary tokens.

The Regularized RLHF Objective

The core innovation of RLHF is combining reward maximization with KL regularization to create a balanced optimization problem.

The Complete Objective:

The RLHF objective function is:

$$J(\theta) = \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi_\theta(\cdot|x)} [r_\phi(y|x) - \beta \cdot D_{\text{KL}}(\pi_\theta(\cdot|x) || \pi_{\text{ref}}(\cdot|x))] \quad (4)$$

Dissecting Equation (4):

This is the equation that ties everything together. Let's understand it term by term:

- $\mathbb{E}_{x \sim \mathcal{D}}$ means we average over prompts x from our training dataset $\mathcal{D}$
- $\mathbb{E}_{y \sim \pi_\theta(\cdot|x)}$ means for each prompt, we sample a response y from our current policy
- $r_\phi(y|x)$ is the reward model score - this tells us how good response y is for prompt x based on human preferences (learned via Bradley-Terry model)
- $D_{\text{KL}}(\pi_\theta(\cdot|x) || \pi_{\text{ref}}(\cdot|x))$ measures how far our policy has drifted from the reference
- $\beta > 0$ is a hyperparameter controlling the strength of regularization - higher β means stronger constraint
- The minus sign on the KL term makes it a penalty - higher divergence reduces the objective

The Tension in the Objective:

This objective creates a deliberate tension:

- The reward term $r_\phi(y|x)$ wants to maximize alignment with human preferences
- The KL penalty $-\beta \cdot D_{\text{KL}}$ wants to minimize deviation from the reference model
- These two goals compete: getting higher reward often requires changing the policy, but changing the policy increases KL
- The coefficient β determines the balance: small β favors reward, large β favors staying close to reference
- Optimization finds responses that are "good enough" on reward while not diverging too much

Expanded Form:

We can rewrite the objective by expanding the KL term:

$$J(\theta) = \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi_\theta(\cdot|x)} \left[r_\phi(y|x) - \beta \log \frac{\pi_\theta(y|x)}{\pi_{\text{ref}}(y|x)} \right] \quad (5)$$

Rearranging:

$$J(\theta) = \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi_\theta(\cdot|x)} [r_\phi(y|x) + \beta \log \pi_{\text{ref}}(y|x) - \beta \log \pi_\theta(y|x)] \quad (6)$$

Why This Form is Illuminating:

Equation (6) reveals three distinct forces:

- $r_\phi(y|x)$: External reward pushing toward human-preferred responses
- $\beta \log \pi_{\text{ref}}(y|x)$: A bonus for generating responses the reference model would generate - this keeps us "grounded"
- $-\beta \log \pi_\theta(y|x)$: An entropy bonus encouraging exploration and diversity in the current policy
- Together, these create a modified reward landscape where we optimize for alignment while staying in familiar territory

The Role of the KL Coefficient β

The hyperparameter β is critical in determining the behavior of RLHF training. Understanding how to set it requires understanding what happens at different scales.

Effect of Different β Values:

- **Small β (e.g., 0.01):**
 - The KL penalty term $\beta \cdot D_{\text{KL}}$ contributes very little to the objective
 - The policy is free to change dramatically to maximize reward
 - Training is fast - reward scores improve quickly
 - Risk: The model may find and exploit reward model bugs, generate repetitive or nonsensical outputs that score high, or forget capabilities from pre-training
 - Example: With $\beta = 0.01$ and $D_{\text{KL}} = 2.0$, the penalty is only 0.02 - negligible compared to typical rewards
- **Moderate β (e.g., 0.1-0.2):**
 - Balanced regularization - this is the typical working range
 - The policy updates carefully, improving reward while respecting the reference
 - Training is steady - reward improves at a measured pace
 - The model maintains diversity and broad capabilities
 - Example: With $\beta = 0.1$ and $D_{\text{KL}} = 2.0$, the penalty is 0.2 - a meaningful fraction of typical reward differences
- **Large β (e.g., 1.0):**
 - Strong regularization - the KL term dominates
 - The policy barely changes from the reference
 - Training is very slow - reward improvements are minimal
 - The model strongly preserves reference behavior, sometimes too much
 - Example: With $\beta = 1.0$ and $D_{\text{KL}} = 2.0$, the penalty is 2.0 - likely larger than reward differences, so the policy won't change much

Choosing β in Practice:

The optimal β depends on:

- How much you trust your reward model (better reward model allows smaller β)
- How much you want to preserve reference behavior (more preservation needs larger β)

- The scale of your rewards (if rewards are large, you need larger β to balance)
- Your tolerance for risk (production systems typically use larger β for safety)

Many practitioners start with $\beta \approx 0.1$ and adjust based on monitoring KL-divergence and reward during training.

Worked Example: KL-Regularized Policy Update

Let's work through a detailed, concrete example showing how KL-divergence affects what the model learns.

Setup:

- Prompt: $x =$ "Explain what a rainbow is"
- KL coefficient: $\beta = 0.2$
- We'll compare three candidate responses

Candidate Responses:

1. y_1 : "A rainbow is an optical phenomenon." (Short, technical)
2. y_2 : "A rainbow is a beautiful arc of colors that appears in the sky when sunlight passes through water droplets, splitting into its component colors." (Detailed, natural)
3. y_3 : "A rainbow is a wonderful, amazing, spectacular, incredible optical phenomenon that creates beautiful colors." (Verbose, repetitive)

Step 1: Reward Model Scores

Suppose the reward model (trained on human preferences) assigns:

$$r_\phi(y_1|x) = 3.0 \quad (\text{correct but brief}) \tag{7}$$

$$r_\phi(y_2|x) = 6.5 \quad (\text{informative and natural}) \tag{8}$$

$$r_\phi(y_3|x) = 7.2 \quad (\text{reward model is fooled by length and positive words}) \tag{9}$$

Step 2: Compute Log-Probabilities

Now we need the probabilities each model assigns. Suppose:

For response y_1 (tokenized as 7 tokens):

$$\log \pi_\theta(y_1|x) = -3.5 \tag{10}$$

$$\log \pi_{\text{ref}}(y_1|x) = -3.2 \tag{11}$$

For response y_2 (tokenized as 22 tokens):

$$\log \pi_\theta(y_2|x) = -11.0 \tag{12}$$

$$\log \pi_{\text{ref}}(y_2|x) = -10.5 \tag{13}$$

For response y_3 (tokenized as 18 tokens):

$$\log \pi_{\theta}(y_3|x) = -6.0 \quad (14)$$

$$\log \pi_{\text{ref}}(y_3|x) = -15.0 \quad (\text{reference finds this very unnatural!}) \quad (15)$$

Step 3: Compute KL Divergence for Each Response

The KL contribution for each response is the log-ratio:

$$D_{\text{KL}}(y_1) = \log \pi_{\theta}(y_1|x) - \log \pi_{\text{ref}}(y_1|x) \quad (16)$$

$$= -3.5 - (-3.2) = -0.3 \quad (17)$$

$$D_{\text{KL}}(y_2) = \log \pi_{\theta}(y_2|x) - \log \pi_{\text{ref}}(y_2|x) \quad (18)$$

$$= -11.0 - (-10.5) = -0.5 \quad (19)$$

$$D_{\text{KL}}(y_3) = \log \pi_{\theta}(y_3|x) - \log \pi_{\text{ref}}(y_3|x) \quad (20)$$

$$= -6.0 - (-15.0) = 9.0 \quad (\text{huge divergence!}) \quad (21)$$

Interpreting the KL Values:

- y_1 and y_2 have small negative KL: the current policy assigns slightly *lower* probability than reference (being conservative)
- y_3 has large positive KL of 9.0: the current policy assigns much *higher* probability than reference
- The reference model thinks y_3 is very unlikely ($\log \pi_{\text{ref}} = -15.0$), but the current policy rates it as reasonably likely ($\log \pi_{\theta} = -6.0$)
- This large divergence is a red flag - the policy is going somewhere unusual

Step 4: Compute Regularized Rewards

Now apply the KL penalty with $\beta = 0.2$:

$$\text{Reward}_{\text{eff}}(y_1) = r_{\phi}(y_1|x) - \beta \cdot D_{\text{KL}}(y_1) \quad (22)$$

$$= 3.0 - 0.2 \times (-0.3) \quad (23)$$

$$= 3.0 + 0.06 = 3.06 \quad (24)$$

$$\text{Reward}_{\text{eff}}(y_2) = r_\phi(y_2|x) - \beta \cdot D_{\text{KL}}(y_2) \quad (25)$$

$$= 6.5 - 0.2 \times (-0.5) \quad (26)$$

$$= 6.5 + 0.10 = 6.60 \quad (27)$$

$$\text{Reward}_{\text{eff}}(y_3) = r_\phi(y_3|x) - \beta \cdot D_{\text{KL}}(y_3) \quad (28)$$

$$= 7.2 - 0.2 \times 9.0 \quad (29)$$

$$= 7.2 - 1.8 = 5.4 \quad (30)$$

Step 5: Compare and Decide

Summary Table:

Response	Raw Reward	KL Penalty	Effective Reward
y_1 (Short)	3.0	-0.06 (bonus)	3.06
y_2 (Natural)	6.5	-0.10 (bonus)	6.60
y_3 (Verbose)	7.2	+1.8 (penalty)	5.4

What Just Happened?

This is the magic of KL regularization:

- Without KL penalty, y_3 would win (raw reward 7.2 vs 6.5)
- The policy would learn to generate verbose, repetitive responses because they fool the reward model
- With KL penalty, y_3 is heavily penalized (effective reward drops from 7.2 to 5.4)
- Now y_2 wins (effective reward 6.60 vs 5.4) - the natural, informative response
- y_1 and y_2 get small bonuses because they're more conservative than the current policy (negative KL becomes positive contribution)

Step 6: The Learning Update

Based on these effective rewards, the gradient update will:

- **Strongly increase** the probability of y_2 (highest effective reward)
- **Moderately increase** the probability of y_3 (still positive, but less so)
- **Slightly increase** the probability of y_1 (positive but small)
- The relative strengths prevent the model from collapsing to reward-hacking behavior
- Over many updates, the policy learns to generate responses like y_2 - natural and informative

What if We Changed β ?

Let's see how different β values would affect y_3 :

- $\beta = 0.05$: Effective reward = $7.2 - 0.05(9.0) = 6.75$ (still wins! reward hacking succeeds)

- $\beta = 0.10$: Effective reward = $7.2 - 0.10(9.0) = 6.30$ (close call, might still win)
- $\beta = 0.20$: Effective reward = $7.2 - 0.20(9.0) = 5.40$ (loses to y_2 , as we saw)
- $\beta = 0.50$: Effective reward = $7.2 - 0.50(9.0) = 2.70$ (heavily penalized)

This shows why tuning β matters - too small and reward hacking succeeds, too large and even good responses get over-penalized.

Why KL-Divergence Prevents Reward Hacking

Let's understand more deeply why KL regularization is so effective at preventing pathological behaviors.

The Core Principle:

The reference policy π_{ref} was trained on high-quality, diverse data through supervised fine-tuning. It has learned what "normal" text looks like. Any response that is very unlikely under π_{ref} is, by definition, abnormal.

How Reward Models Get Exploited:

Reward models learn from finite human preference data. They can develop spurious correlations:

- "Longer responses are better" (humans prefer detail, but not excessive verbosity)
- "Positive words score high" (humans like positivity, but not fake enthusiasm)
- "Formal language is preferred" (humans appreciate clarity, but not stuffiness)

Without KL penalty, the policy can exploit these patterns:

- Generate extremely long responses to maximize length bonus
- Stuff responses with positive adjectives ("amazing, wonderful, incredible...")
- Use overly formal or technical language even when inappropriate

How KL Saves Us:

All of these exploitative behaviors have one thing in common: they're very unlikely under the reference policy. The reference model, trained on normal text, would rarely generate such responses. Therefore:

- Exploitative response: $\pi_{\text{ref}}(y_{\text{exploit}}|x)$ is very small
- This makes $\log \pi_{\text{ref}}(y_{\text{exploit}}|x)$ very negative (large magnitude)
- If the policy assigns reasonable probability: $\log \pi_{\theta}(y_{\text{exploit}}|x)$ is moderate
- KL divergence $\log \pi_{\theta}(y_{\text{exploit}}|x) - \log \pi_{\text{ref}}(y_{\text{exploit}}|x)$ is large and positive
- The KL penalty $\beta \cdot D_{\text{KL}}$ wipes out the spurious reward gain

The reference policy acts as a "sanity check" - it has seen lots of good text and knows what normal looks like, even if it's not perfectly aligned.

Connection to Trust Regions

KL-divergence regularization is closely related to the trust region methods used in reinforcement learning optimization.

The Trust Region Concept:

A trust region is a neighborhood around the current policy where our estimates (reward model, value function) are reliable. The idea is simple:

- Our reward model r_ϕ was trained on responses from a certain distribution
- If the policy moves too far from that distribution, the reward model’s predictions become unreliable
- We should only trust updates that keep us within a "region of trust"
- KL-divergence provides a natural way to define this region

Mathematical Connection:

The KL penalty enforces a soft trust region constraint. Instead of a hard constraint:

$$\text{maximize } \mathbb{E}[r_\phi(y|x)] \quad \text{subject to} \quad D_{\text{KL}}(\pi_\theta || \pi_{\text{ref}}) \leq \delta$$

we use the penalized objective:

$$\text{maximize } \mathbb{E}[r_\phi(y|x) - \beta \cdot D_{\text{KL}}(\pi_\theta || \pi_{\text{ref}})]$$

These are related by Lagrangian duality - the penalty coefficient β corresponds to the Lagrange multiplier for the constraint. The soft version is easier to optimize because it’s differentiable everywhere.

Practical Implementation Considerations

Computing KL in Practice:

For a generated sequence, we compute the KL contribution as:

$$D_{\text{KL}} = \sum_{t=1}^T [\log \pi_\theta(y_t|x, y_{<t}) - \log \pi_{\text{ref}}(y_t|x, y_{<t})] \quad (31)$$

Why This Formula Works:

- We sum over only the actual tokens generated, not all possible tokens in the vocabulary
- This is equivalent to the full KL when we’re computing it for a specific sampled sequence
- Both models see the same prompt x and context $y_{<t}$ at each step
- The reference model π_{ref} is frozen - we don’t update its parameters
- We only need a forward pass through π_{ref} to get its log-probabilities

Monitoring During Training:

Key metrics to track:

- **Mean KL per token:** Typically should stay between 0.01 and 0.1
- **Maximum KL per sequence:** Watch for outliers that diverge too far
- **KL vs Reward correlation:** Are we getting reward improvements at reasonable KL cost?
- **Token-position KL:** Does divergence grow later in sequences? (common pattern)
 - If mean KL grows too large (> 0.2 per token), consider:
 - Increasing β to strengthen regularization
 - Reducing learning rate to make updates more conservative
 - Checking if the reward model is giving unreliable signals
 - If mean KL is too small (< 0.001 per token), the policy may not be improving enough - consider decreasing β .

Mathematical Properties of KL-Divergence

Understanding the mathematical properties helps us reason about optimization behavior.

Non-Negativity and Identity of Indiscernibles:

For any distributions P and Q :

$$D_{\text{KL}}(P||Q) \geq 0 \quad (32)$$

with equality if and only if $P = Q$ almost everywhere.

Why This Matters:

- The KL penalty is always pulling toward zero (where $\pi_\theta = \pi_{\text{ref}}$)
- We can never get a "negative penalty" by diverging more
- The minimum KL of zero acts as an anchor point
- Any policy update that changes π_θ will increase KL (unless it makes π_θ closer to π_{ref})

Convexity:

KL-divergence is convex in its first argument. For any $\lambda \in [0, 1]$:

$$D_{\text{KL}}(\lambda P_1 + (1 - \lambda)P_2||Q) \leq \lambda D_{\text{KL}}(P_1||Q) + (1 - \lambda)D_{\text{KL}}(P_2||Q) \quad (33)$$

Implications for Optimization:

- The regularized objective $r(y) - \beta \cdot D_{\text{KL}}$ has nice optimization properties
- Gradient descent won't get stuck in local minima from the KL term alone
- Averaging policies (mixture of policies) has KL no worse than the weighted average of individual KLs
- This supports ensemble methods and multi-stage training approaches

Pinsker's Inequality:

KL-divergence upper bounds the total variation distance:

$$\text{TV}(P, Q) = \frac{1}{2} \sum_x |P(x) - Q(x)| \leq \sqrt{\frac{1}{2} D_{\text{KL}}(P||Q)} \quad (34)$$

What This Tells Us:

- If we keep KL small, the distributions can't differ too much in worst-case pointwise probability
- Controlling KL automatically controls how different the policies are overall
- Even if one specific response has large probability ratio, if KL is bounded, most responses must have similar probabilities
- This gives us confidence that behavior won't change drastically

Summary

KL-divergence regularization is fundamental to stable and effective RLHF:

- **Measures distributional difference** between fine-tuned policy π_θ and reference policy π_{ref}
- **Prevents pathological behaviors** including reward hacking, mode collapse, and catastrophic forgetting
- **Controlled by coefficient β** which balances alignment (reward maximization) and preservation (staying close to reference)
- **Acts as a trust region** keeping the policy in regions where the reward model is reliable
- **Provides theoretical guarantees** through convexity and information-theoretic bounds

The Key Insight:

Unbounded optimization of an imperfect reward model leads to exploitation and failure. By constraining the policy to stay near a reference that was trained on high-quality data, we ensure that improvements are genuine rather than exploitative. KL-divergence provides the mathematical language to formalize this constraint as a differentiable penalty term.

The elegance of KL regularization is that it's automatic - we don't need to manually specify what "bad" outputs look like. Anything the reference model finds very unlikely is automatically penalized, providing a built-in sanity check based on what the model learned during pre-training.

Practical Takeaways:

- **Start conservative:** Begin with $\beta \approx 0.1$ to 0.2 for stable training
- **Monitor KL:** Track mean KL per token (target: 0.01-0.1) and watch for concerning trends

- **Balance objectives:** The art of RLHF is finding the right β that allows meaningful improvement while preventing exploitation
- **Trust the math:** KL-divergence has strong theoretical properties that make it reliable for this purpose
- **Remember the reference:** The quality of π_{ref} matters - it should be a well-trained SFT model that represents "normal, good" behavior

References

1. Kullback, S., & Leibler, R. A. (1951). On information and sufficiency. *The Annals of Mathematical Statistics*, 22(1), 79-86.
2. Schulman, J., et al. (2017). Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*.
3. Ouyang, L., et al. (2022). Training language models to follow instructions with human feedback. *Advances in Neural Information Processing Systems*, 35.
4. Ziegler, D. M., et al. (2019). Fine-tuning language models from human preferences. *arXiv preprint arXiv:1909.08593*.
5. Stiennon, N., et al. (2020). Learning to summarize from human feedback. *Advances in Neural Information Processing Systems*, 33.
6. Jaques, N., et al. (2019). Way off-policy batch deep reinforcement learning of implicit human preferences in dialog. *arXiv preprint arXiv:1907.00456*.